Intro to Deep Learning

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

Deep Learning: Intro

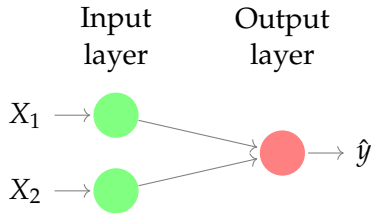
- Linear Models may miss the nonlinearities that best approximate $f^*(x)$
- Neural networks are simple models.
- The model has **linear combinations** of inputs that are passed through **nonlinear activation functions** called nodes (or, in reference to the human brain, neurons).

Deep Learning: Intro

- Let's start with a familiar and simple model, the linear model

Deep Learning: Intro

- Let's start with a familiar and simple model, the linear model



Single Layer Neural Networks

- A neural network takes an input vector of p variables

$$X = (X_1, X_2, ..., X_p) \tag{1}$$

- and builds a nonlinear function $f(X)$ to predict the response y .

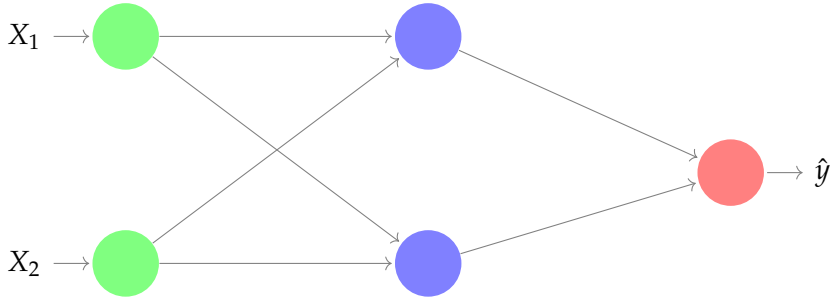
$$y = f(X) + u \tag{2}$$

- The Single layer NN model has the form

$$f(X) = f^{(output)}(g(X)) \tag{3}$$

- where g is the activation function in the hidden unit
- the second layer, $f^{(output)}$ is the output layer of the network

Single Layer Neural Networks



Single Layer Neural Networks

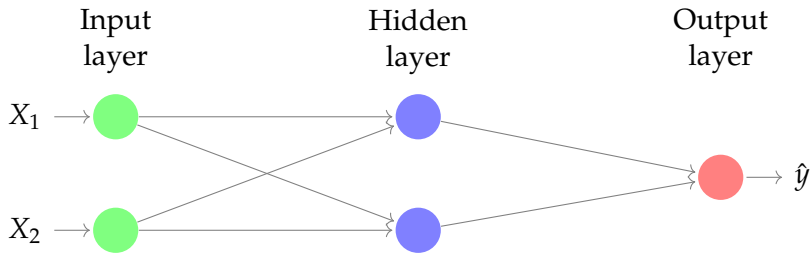
- NN are made of **linear combinations** of inputs that are passed through **nonlinear activation functions**

Worked Example I: Single Layer Neural Networks

- 2 Predictors: $p = 2$, $X = (X_1, X_2)$
- 2 Nodes: $K = 2$, $A_1(X)$ and $A_2(X)$
- Non-linear activation function $g(z) = z^2$
- Want to predict a number $\in \mathbb{R}$: identity output function ($f^{(output)} : \mathbb{R} \to \mathbb{R}$ such that $f^{(output)}(x) = x$)

Worked Example I: Single Layer Neural Networks

- 2 Predictors: $p = 2$, $X = (X_1, X_2)$
- 2 Nodes: $K = 2$, $A_1(X)$ and $A_2(X)$
- Non-linear activation function $g(z) = z^2$
- Want to predict a number $\in \mathbb{R}$: identity output function ($f^{(output)} : \mathbb{R} \to \mathbb{R}$ such that $f^{(output)}(x) = x$)



Worked Example I: Single Layer Neural Networks

- 2 Predictors: $p = 2$, $X = (X_1, X_2)$
- 2 Nodes: $K = 2$, $A_1(X)$ and $A_2(X)$
- Non-linear activation function $g(z) = z^2$
- Want to predict a number $\in \mathbb{R}$: identity output function ($f^{(output)} : \mathbb{R} \to \mathbb{R}$ such that $f^{(output)}(x) = x$)

$$f(X) = f^{(output)}(g(X)) \tag{4}$$

Worked Example I: Single Layer Neural Networks

- 2 Predictors: $p = 2$, $X = (X_1, X_2)$
- 2 Nodes: $K = 2$, $A_1(X)$ and $A_2(X)$
- Non-linear activation function $g(z) = z^2$
- Want to predict a number $\in \mathbb{R}$: identity output function ($f^{(output)} : \mathbb{R} \to \mathbb{R}$ such that $f^{(output)}(x) = x$)

$$f(X) = f^{(output)}(g(X)) \tag{4}$$

$$f(X) = \beta_0 + \sum_{k=1}^{2} \beta_k \left(w_{k0} + \sum_{j=1}^{2} w_{kj} X_j \right)^2 \tag{5}$$

Why not linear activation functions?

Worked Example II : The "Exclusive OR (XOR)" Function

- The exclusive disjunction of a pair of propositions, (p, q), is supposed to mean that p is true or q is true, but not both
- It's truth table is:

q	p	y
0	0	0
0	1	1
1	0	1
1	1	0

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

Activation Functions

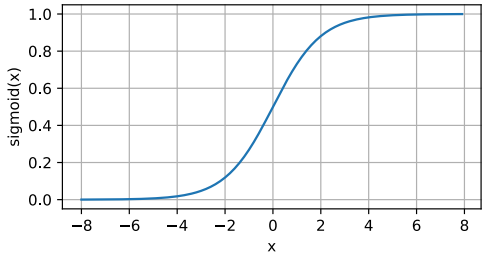
Sigmoid Function (Logit)

- The sigmoid function transforms its inputs, for which values lie in the domain $\mathbb{R}$, to outputs that lie on the interval $(0, 1)$.
- For that reason, the sigmoid is often called a squashing function: it squashes any input in the range (-inf, inf) to some value in the range $(0, 1)$:

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

Activation Functions

Sigmoid Function (Logit)



- When attention shifted to gradient based learning, the sigmoid function was a natural choice because it is smooth and differentiable.

Activation Functions

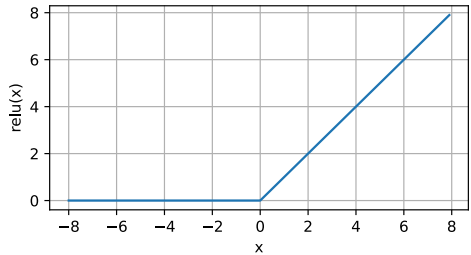
ReLU Function

- ReLU Function
  - The most popular choice, due to both simplicity of implementation and its good performance on a variety of predictive tasks, is the rectified linear unit (ReLU).
  - ReLU provides a very simple nonlinear transformation. Given an element x, the function is defined as the maximum of that element and 0:

$$\text{ReLU}(x) = \max\{x, 0\}$$

Activation Functions

- ReLU function retains only positive elements and discards all negative elements by setting them to 0.
- It is piecewise linear.



Neural Networks: Activation Functions

- $\text{Sigmoid}(x) = \frac{1}{1+\exp(-x)}$
- $\text{ReLU}(x) = \max\{x, 0\}$
- Among others (see more here)
- Hidden unit design remains an active area of research, and many useful hidden unit types remain to be discovered

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

Output Functions

- The choice of output unit is related to the problem at hand
  - Regression
  - Classification
    - Binary
    - Multiclass
- La output function determina qué pérdida (loss) tiene sentido.

Output y pérdida: el match correcto

Problema	Output $f(\cdot)$	Pérdida L
Regresión	identidad	MSE
Clasif. binaria	sigmoide	cross-entropy
Clasif. multiclase	softmax	cross-entropy

- Cada pérdida sale de la log-verosimilitud del modelo probabilístico correspondiente.
- *No* se mezclan: no tiene sentido usar MSE con softmax, ni cross-entropy con identidad.

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

Training the network

- El objetivo es

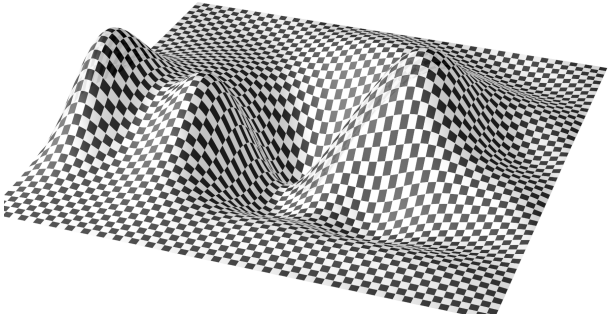
$$\hat{f} = \arg\min_{f} \left\{ \sum_{i=1}^{n} L\big(y_i, f(X_i; \Theta)\big) \right\} \tag{6}$$

- Para una Single Layer NN:

$$f(X, \beta, w) = f\left[\beta_0 + \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j \right) \right] \tag{7}$$

- Los parámetros a estimar son $\Theta = (w, \beta)$.

Training the network



- $L(.)$ es una superficie en un espacio de altas dimensiones.
- Queremos llegar al *mínimo* (o cerca).
- A diferencia de OLS, la superficie ya **no es convexa**: puede haber mínimos locales, puntos silla, valles.
- No hay fórmula cerrada: hay que **bajar paso a paso**.

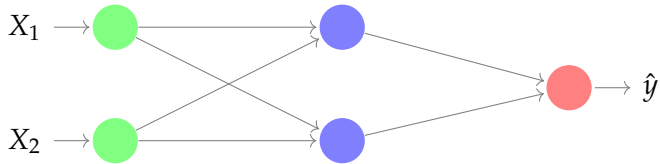
Training the network

Example: House Prices



Training the network

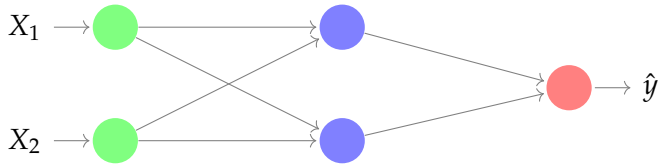
Example: House Prices



- **Hidden layer** (sigmoide):
  - $A_1 = \sigma(w_{11}X_1 + w_{12}X_2 + w_{10})$
  - $A_2 = \sigma(w_{21}X_1 + w_{22}X_2 + w_{20})$
- **Output** (identidad): $\hat{y}_i = \beta_0 + \beta_1 A_1 + \beta_2 A_2$
- **Pérdida**: MSE $= \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$

Training the network

Backpropagation



Training the network

Backpropagation

- La idea completa:
  1. **Forward pass**: calcular $\hat{y}$ y $\mathcal{L}$ dado Θ actual.
  2. **Backward pass**: calcular $\nabla_\Theta \mathcal{L}$ usando la regla de la cadena de *atrás hacia adelante*, reutilizando productos intermedios.
  3. **Update**: $\Theta \leftarrow \Theta - \epsilon \nabla_\Theta \mathcal{L}$.
  4. Repetir.
- La magia es que el costo computacional de backprop es *del mismo orden* que el forward pass. (Rumelhart, Hinton & Williams, 1986.)

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

Batch Gradient Descent

- El gradiente se calcula **usando todo el dataset** en cada paso:

$$\theta' = \theta - \epsilon \cdot \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta L(y_i, f(X_i; \theta))$$

- Trayectoria *suave*; cada paso es una estimación insesgada del gradiente verdadero.
- Terriblemente lento en datasets grandes: cada paso cuesta $O(n)$.
- Sin embargo, escala bien con el *número de variables* p: mejor que Normal Equations cuando p es enorme.

Stochastic Gradient Descent (SGD)

- En cada paso, SGD toma **una observación aleatoria** y calcula el gradiente sólo con ella:

$$\theta' = \theta - \epsilon \cdot \nabla_\theta L(y_i, f(X_i; \theta)), \quad i \sim \text{Unif}\{1, \ldots, n\}$$

- Cada paso es $O(1)$.
- Trayectoria ruidosa (zig-zag visible en las curvas de nivel).

Mini-batch Gradient Descent

- Punto medio entre Batch y SGD: en cada paso usamos un **mini-batch** de tamaño B:

$$\theta' = \theta - \epsilon \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta L(y_i, f(X_i; \theta))$$

- Típicamente 2^x eg $\{32, 64, 128, 256\}$.
- **Ventajas**:
  - Compromiso: menos ruido que SGD, mucho más barato que Batch.
  - Mapea bien sobre GPUs (paralelizable dentro del batch).
  - El ruido residual ayuda a escapar de mínimos locales.

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

¿Por qué necesitamos más que SGD?

- Problemas de SGD puro:
  - Aprende *lento* en direcciones de poca curvatura.
  - Oscila en direcciones de mucha curvatura.
  - Misma learning rate para todos los parámetros, sin importar su escala.
- Soluciones históricas, en orden:
  1. **Momentum** (Polyak, 1964): memoria del gradiente pasado.
  2. **RMSProp** (Tieleman & Hinton, 2012): learning rate adaptativa por parámetro.
  3. **Adam** (Kingma & Ba, 2014): Momentum + RMSProp.

SGD + Momentum (Polyak, 1964)

$$v' = \gamma v - \epsilon \cdot \nabla_\theta \mathcal{L}(\theta)$$
$$\theta' = \theta + v'$$

- Agrega una "inercia" al gradiente.
- Acumula dirección $\Rightarrow$ suaviza la trayectoria.
- Ayuda a escapar de mínimos poco profundos y acelera en direcciones consistentes.
- $\gamma \in [0, 1)$, típicamente 0.9.
- Lee el estado v como un *momentum físico*: el parámetro tiene inercia.

RMSProp (Tieleman & Hinton, 2012)

$$E[\nabla_\theta \mathcal{L}^2]_t = \rho\, E[\nabla_\theta \mathcal{L}^2]_{t-1} + (1-\rho)\,(\nabla_\theta \mathcal{L})^2$$

$$\theta' = \theta - \frac{\epsilon}{\sqrt{E[\nabla_\theta \mathcal{L}^2]_t + \eta}} \cdot \nabla_\theta \mathcal{L}$$

- Mantiene una *media móvil* del gradiente al cuadrado.
- Divide el gradiente por su magnitud típica reciente:
  - Parámetros con gradientes grandes $\Rightarrow$ pasos chicos.
  - Parámetros con gradientes chicos $\Rightarrow$ pasos grandes.
- **Learning rate adaptativa por parámetro**. Ya no todos avanzan al mismo ritmo.

Adam (Kingma & Ba, 2014)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L} \qquad \text{(momento 1, estilo Momentum)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \mathcal{L})^2 \qquad \text{(momento 2, estilo RMSProp)}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \text{(corrección de sesgo)}$$

$$\theta' = \theta - \epsilon \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \eta}$$

- Combina: **Momentum + RMSProp**.
- La corrección de sesgo es clave al inicio (cuando m, v arrancan en 0).
- **Default moderno** en casi todo DL aplicado: robusto, rápido, poco sensible a ϵ.

¿Cuál usar? RMSProp vs Adam vs SGD

Zhou et al. (2020)

- Aunque Adam converge **más rápido**, SGD suele encontrar soluciones que **generalizan mejor**.
- Zhou et al. (2020) modela ambos métodos teóricamente:
  - **SGD**: el ruido del gradiente estocástico actúa como regularización implícita.
    - Escapa más fácil de mínimos locales (flat vs sharp minima).
    - Alcanza soluciones con mejor generalización.
  - **Adam**: puede quedar atrapado en mínimos subóptimos (sharp minima).
- El ruido inherente al SGD actúa como una forma de regularización que favorece la generalización.
- En la práctica a veces se arranca con Adam (converge rápido), y para el *fine-tuning* final se cambia a SGD con momentum.

Agenda

1. Single Layer Neural Networks
2. Activation Functions
   - Output Functions
3. Training the network
4. Batch, Stochastic, Mini-batch
5. Optimizadores
6. Recap

Recap

1. Una Single Layer NN es una combinación lineal seguida de una activación no-lineal.
2. La no-linealidad de la activación es la clave: es donde sucede ”la magia”
3. La activación de salida se elige según el problema (identidad / sigmoide / softmax).
4. Entrenamos con backpropagation + gradiente descendente.
5. Tres formas de estimar el gradiente: Batch, SGD, Mini-batch.
6. Optimizadores modernos: Momentum, RMSProp, Adam. El ruido del SGD ayuda a generalizar.

Next

- Teorema de aproximación universal.
- Arquitecturas profundas (multicapa).
- Regularización: dropout, early stopping.
- Aplicación clasificación multiclase: MNIST.